github readme

```
# VoiceBudget Bot ⌧⌧
Telegram voice-first expense tracker & budget coach powered by GPT-4o

> Talk to a bot about your spending. It logs everything and sends you a simple monthly r

---

## ✨ Features

- **Voice logging** – Send a voice message like
  `“Today I spent 80k on lunch and 30k on taxi”`
  The bot transcribes it, extracts each expense (amount, category, date, note), and save

- **Text logging** – Type messages like:
  - `120000 uzs groceries`
  - `$15 Uber`
  - `Paid 50k for coffee`
  and they are parsed into structured expenses automatically.[web:33][web:39][web:45]

- **Daily check-in** – At your chosen time, the bot asks:
  `“How was your spending today?”`
  Reply with voice or text, and it logs all expenses for the day.[web:21][web:39]

- **Quick summaries**
  - `/today` – today’s total & categories
  - `/week` – last 7 days
  - `/month` – current month[web:33][web:45]

- **Monthly report** – On the 1st of each month you get:
  - Total spent
  - Top categories
  - Comparison vs last month
  - 2–3 concrete suggestions on how to improve[web:32][web:35][web:45]

- **Auto-categorization** – GPT-4o guesses categories like Food, Transport, Groceries, S

- **Multi-device** – Works anywhere Telegram works: phone, desktop, web.[web:21][web:33]

> Optional future features: budgets & alerts, photo receipts (OCR), exports, and deeper

---

## ⌧ How it works

High-level flow:
```

```
1. You send a **voice** or **text** message to the bot.
2. For voice:
   - Bot downloads the audio from Telegram.[web:42]
   - Sends it to OpenAI speech-to-text (Whisper / audio endpoint) to get the transcript.
3. The transcript (or text message) is sent to **GPT-4o** with a strict prompt to extrac
4. The backend parses the JSON and saves rows to the database / Google Sheet.[web:21][we
5. When you call `/today`, `/week`, `/month`, or when the month ends, the bot queries the

---

## ▨ Data model (simplified)

**Users**

- `id` – internal id
- `telegram_user_id`
- `username`
- `currency` – e.g. `UZS`, `USD`
- `timezone` – e.g. `Asia/Tashkent`
- `daily_reminder_enabled` – boolean
- `daily_reminder_time` – `HH:MM`

**Expenses**

- `id`
- `user_id`
- `created_at` – timestamp
- `transaction_date` – date
- `amount` – number
- `currency` – string
- `category` – `food | transport | groceries | shopping | bills | subscriptions | other`
- `description` – text
- `source` – `voice | text | photo`
- `raw_input` – original message/transcript

---

## ▨ Getting started

### 1. Create a Telegram bot

1. Open Telegram and start a chat with **@BotFather**.
2. Run `/newbot` and follow the steps.
3. Copy the **bot token** (you'll use this as an environment variable).[web:44]

### 2. Set up the backend (example: Python stack)

> You can adapt this to Node or a low-code tool like n8n if you prefer.[web:21][web:43]

1. Clone this repository:

   ```bash
   git clone https://github.com/yourname/voicebudget-bot.git
   cd voicebudget-bot
```

2. Create and activate a virtualenv, then install dependencies:
```

```
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

3. Create a `.env` file:

```
TELEGRAM_BOT_TOKEN=your_telegram_token_here
OPENAI_API_KEY=your_openai_api_key_here
DATABASE_URL=postgres://user:pass@host:port/dbname   # or use SQLite/Sheets
BASE_URL=https://your-server.com  # for webhooks
```

4. Run database migrations / create tables as needed.
5. Set the Telegram webhook to point to your server:

```
curl -X POST "https://api.telegram.org/bot<TELEGRAM_BOT_TOKEN>/setWebhook" \
  -d "url=<BASE_URL>/webhook/telegram"
```

6. Start your server:

```
python app.py
```

Main endpoints (example)

- `POST /webhook/telegram`
  Handles all Telegram updates.
  - If message has **voice** → download audio → transcribe → parse expenses → save → reply.[web:21][web:42][web:45]
  - If message has **text** → parse text → parse expenses → save → reply.[web:33][web:39]
- Scheduled jobs (cron / worker):
  - Daily: send check-in message to users with daily reminders enabled.[web:21][web:39]
  - Monthly: generate and send monthly reports.[web:32][web:35][web:45]

GPT-4o prompts (examples)

Expense extraction

System prompt

> You are an assistant that extracts personal expenses from short messages. Always respond with valid JSON only. Do not include explanations or any extra text.[web:21][web:39]

User prompt (template)

```
Text: "Today I spent 80k on food and 30k on taxi and 10 dollars on Netflix."
Base currency: UZS
Current date: 2026-03-20

Return JSON with this schema:
{
  "expenses": [
    {
      "amount": number,
      "currency": string,
      "category": string,
      "description": string,
      "date": "YYYY-MM-DD"
    }
  ]
}
```

The model is expected to output something like:

```
{
  "expenses": [
    {
      "amount": 80000,
      "currency": "UZS",
      "category": "food",
      "description": "food",
      "date": "2026-03-20"
    },
    {
      "amount": 30000,
      "currency": "UZS",
      "category": "transport",
      "description": "taxi",
      "date": "2026-03-20"
    },
    {
      "amount": 10,
      "currency": "USD",
      "category": "subscriptions",
      "description": "Netflix",
      "date": "2026-03-20"
    }
  ]
}
```

Monthly report

System prompt

> You are a friendly budget coach. Given a user's monthly spending stats, explain clearly where their money went and suggest 2–3 practical actions to improve. Keep your answer under 200 words.[web:32][web:35]

The backend passes a JSON summary like:

```
{
  "month": "2026-03",
  "total_spent": 3200000,
  "currency": "UZS",
  "categories": [
    { "name": "food", "amount": 1300000 },
    { "name": "transport", "amount": 500000 },
    { "name": "other", "amount": 1400000 }
  ],
  "comparison": {
    "previous_month_total": 2800000
  }
}
```

GPT-4o returns a short, human-friendly summary that the bot sends to the user.[web:32][web:35][web:45]

Roadmap

- [ ] Budgets per category + alerts when you hit 80%[web:32][web:35]
- [ ] Photo receipt support with OCR[web:21][web:39][web:43]
- [ ] CSV / Notion export[web:22][web:25][web:45]
- [ ] Advanced Q&A about spending patterns ("coffee last month?", "subscriptions per month?")[web:21][web:33][web:39]

⚠ Disclaimer

This bot is for **personal finance tracking only** and does not provide professional financial advice. Always double-check important financial decisions with a qualified expert.

License

Specify your license here (e.g. MIT).

```
If you want, I can now sketch a minimal `app.py` structure (routes + handlers) that matche
```